ElasticMLA: Context-Aware Token-Wise Latent Capacity Allocation for Multi-Head Latent Attention

Anonymous authors
Paper under double-blind review

Abstract

Multi-head Latent Attention (MLA) shrinks the key-value cache by storing a compressed latent per token, but gives every token the same latent width. We treat token-wise latent width as a basis-dependent rate allocation problem: cache bytes are exactly affine in the summed token rates, and upper-tail future-loss risk induces a provably ordered spectrum of safe rates between a token's average and worst reuse offset. Measured on 30M-, 122M- and 250M-parameter MLA models, the required rate rises from 7-9% of the latent width at the mean criterion to 72-74% at the worst offset, with a nearly scale-invariant tail-capacity premium (0.646-0.651); the gap comes from pervasive cancellation across the reuse horizon, not rare catastrophic tokens. The public, production-trained DeepSeek-V2-Lite reproduces the spectrum (8.2% to 78.7%, premium 0.70) and the cancellation signature. A basis ablation shows that in our small models rotating the latent basis cuts the worst-offset requirement by about 50 points, but in DeepSeek-V2-Lite by only 11, so the tail requirement is real while the large basis effect is specific to our models. We also implement a variable-width packed MLA cache and a contextual router, and evaluate the router under pre-registered, audited protocols. It beats exact-byte random allocation at 30M and 122M, fails with a coarse tier grid at 250M and succeeds with a finer one, a tier-resolution confound we diagnose rather than a scale trend. Held to exactly its bytes, it beats position and token-type heuristics (except with the coarse 250M grid) but only ties a per-token identity lookup table, so we do not claim that context beyond the current token helps. On a T4 GPU the packed cache stores 59.6-67.4% of full-MLA cache bytes with essentially unchanged peak memory and a 2.4x decode-latency cost.

1 Introduction

Autoregressive transformer inference stores a key and value representation for each past token in each attention layer. This KV cache grows linearly with context length, batch size, layer count, and attention width, and increasingly constrains serving capacity. Multi-head Latent Attention (MLA), introduced in DeepSeek-V2 [2] and retained in DeepSeek-V3 [3], addresses this cost by caching a shared low-dimensional latent representation and reconstructing per-head content keys and values when needed. Related work converts pretrained multi-head or grouped-query attention models to MLA [4] and combines low-rank attention with layer sharing or quantization [5].

Existing MLA designs nevertheless use one fixed latent width for all tokens. This is conservative if difficult tokens are rare: the width must accommodate tail cases even when most cached states can tolerate stronger compression. Conversely, naïve average-capacity routing can fail because a small number of under-provisioned tokens may affect many future predictions. This tension raises three questions:

1. Does token-level latent capacity vary under a future-loss criterion, and does the pattern replicate across model scales?
2. Can a router use contextual hidden states to place capacity better than a noncontextual policy at exactly the same persistent-cache tensor-payload byte budget?

3. Can token-wise ranks be represented as actual packed storage rather than only simulated by a dense mask?

We answer these questions with **ElasticMLA**, a correctness-first prototype for token-specific MLA latent widths. The core design preserves a full-width first layer, predicts one discrete tier from the resulting contextual state, and shares the selected tier across downstream layers. This makes the routing decision contextual while avoiding the circular requirement of compressing the state needed to decide its own capacity.

Our contributions are:

- a genuine variable-width packed latent cache with token-level ranks and verified dense/full-rank equivalence;
- a formal rate-allocation theory (Section 3.2) proving an exact cache-byte identity and a risk-capacity ordering across upper-tail loss criteria, and a corrected, provenance-tracked future-loss intervention that measures the resulting risk-capacity spectrum at three scales spanning an 8x parameter range (30M-250M);
- a joint-rollout straight-through router objective that optimizes hard deployment tiers while regularizing expected rank;
- a pre-specified 24-sequence-per-scale confirmation (untouched except for the 250M-fine rerun) against exact-byte static and matched-histogram shuffle controls at four scale/tier-grid configurations, together with a diagnostic experiment that isolates tier granularity from model scale as a cause of one configuration's failure; and
- a comparison against simple, non-learned causal heuristics that the random/shuffle controls alone would miss, plus a direct GPU benchmark of persistent cache bytes, peak memory, and decode latency.

The risk-capacity spectrum (contribution 2) is the paper's most robust finding: it is scale- consistent and does not involve the router. The routing contribution is narrower and more mixed, and we report it as such rather than rounding it up: contextual allocation beats *random* matched-budget placement at 30M and 122M, and at 250M once tier resolution is matched to 122M's scheme. At equal bytes it also beats position- and type-based *heuristic* placement, but it only ties a per-token identity lookup table, so its measured advantage is not evidence that context beyond the current token matters. We do not claim optimized serving speed, lower peak memory (the measured reduction is under 1%, Section 5.5), or a reliably calibrated quality constraint at every scale.

2 Background and Related Work

2.1 Multi-head Latent Attention

For hidden state h_t, MLA computes a compressed KV latent

$$c_t = W_{DKV} h_t \in \mathbb{R}^{d_c},$$

then reconstructs content keys and values using $W_{UK} c_t$ and $W_{UV} c_t$. A separate rotary key $k_t^R \in \mathbb{R}^{d_R}$ carries positional information. A fixed-width MLA cache stores (c_t, k_t^R) instead of per-head K/V tensors. For sequence length T, L layers, and element size b, its leading storage term is

$$M_{\text{MLA}} = bTL(d_c + d_R).$$

DeepSeek-V2 reports substantial KV-cache reduction and throughput gains from this representation [2]. MHA2MLA [4] and TransMLA [6] study converting existing architectures to MLA. These works motivate latent compression but do not directly establish token-wise variable-width storage.

2.2 KV-cache compression and adaptive allocation

KV-cache compression includes eviction, quantization, low-rank approximation, cross-layer sharing, and sparse attention. Cross-Layer Latent Attention combines several axes and reports aggressive compression [5]. Such methods show that cache states contain redundancy, but a global compression rate can conceal variation across positions. ElasticMLA differs by allocating a nested latent prefix to every token while retaining all token positions.

Dynamic allocation is challenging because quality depends jointly on many cached states. An isolated token intervention gives useful sensitivity labels but is not a joint-rollout oracle. Our early supervised routers exposed this mismatch: high accuracy on isolated mean-risk labels collapsed toward the smallest tier during simultaneous compression. This negative result motivates direct optimization under the deployed hard-routing rollout.

2.3 Relation to token-conditioned and layer-wise MLA variants

Closest in spirit is EG-MLA [7], which gates latent content per token via an embedding-conditioned mechanism, and CARE [8], which allocates rank across layers using covariance-aware decomposition. ElasticMLA differs along three axes simultaneously: (i) it changes the *persistent cache width* itself through a genuinely packed, ragged representation rather than a fixed-width gate; (ii) the routing decision is trained end-to-end with a straight-through hard-tier joint-rollout surrogate under an explicit rate penalty, rather than a fixed heuristic or layer-only allocation; and (iii) we evaluate it against random, shuffled, *and* causal-heuristic exact-byte controls, exposing a result those comparisons alone would miss: the router ties a per-token identity rule at equal bytes (Section 5.4). We see this combination — runtime per-token packed rate allocation, joint hard-routing optimization, and honest heuristic comparison — as the paper's main empirical novelty, while acknowledging that "learn an importance signal, then spend a discrete resource budget nonuniformly" is a general pattern shared with adaptive-computation and mixture-of-experts routing more broadly.

3 Method

3.1 Nested latent prefixes and packed storage

For each layer ℓ, calibration produces a permutation π_ℓ of the d_c latent channels using gradient-times-activation importance. A token assigned rank r_t stores the nested prefix

$$\tilde{c}_{\ell,t} = c_{\ell,t}[\pi_\ell(1 : r_t)].$$

The persistent representation contains a values buffer, int32 offsets, int16 ranks, an int16 channel permutation, and the decoupled RoPE key. For the evaluated batch-one configurations, its exact cache tensor-payload byte count is

$$M_{\text{packed}} = b\,[Td_c + (L-1)T\bar{r} + TLd_R] + 4L(T+1) + 2LT + 2Ld_c,$$

where $\bar{r} = T^{-1}\sum_t r_t$. The final three terms count int32 offsets, int16 ranks, and int16 channel permutations. The implementation uses $b = 4$. This excludes allocator rounding, framework-object overhead, router weights, and workspace memory. During attention, the prototype reconstructs a dense temporary latent tensor; the metric is therefore neither process/device memory nor a peak-memory or kernel-latency measurement.

3.2 Terminology and two propositions

We deliberately avoid calling r_t an intrinsic matrix rank. Because MLA's latent coordinates can be rotated while compensating the up-projections ($c \mapsto Qc$, $W_U \mapsto W_U Q^{-1}$ for invertible Q), a coordinate-wise nested prefix is **basis dependent**: full-model equivalence under Q does not imply prefix-code equivalence.

This basis dependence echoes prior evidence that MLA's latent coordinates separate reusable content from position-specific information in a way that is sensitive to how the latent basis is analyzed [9]. We therefore call r_t an operational retained- coordinate rate in the calibrated nested codebook $P_{\ell,r_1} \preceq \cdots \preceq P_{\ell,d_c} = I$.

Proposition 1 (exact budget affinity). From the byte formula above, for any two per-sequence rate allocations $\mathbf{r}, \mathbf{s}$ with the same downstream rank sum, $M_{\text{packed}}(\mathbf{r}) = M_{\text{packed}}(\mathbf{s})$; more generally $M_{\text{packed}}(\mathbf{r}) - M_{\text{packed}}(\mathbf{s}) = b(L-1)\sum_t (r_t - s_t)$. This licenses exact-byte controls: any two allocations with equal rank sums have identical cache tensor-payload bytes.

Let $\Delta_{t,j}(r) = \ell_{t+j}(P_r c_t) - \ell_{t+j}(c_t)$ be the signed loss change at future offset $j \in \{0, \ldots, H-1\}$ from truncating only source token t to rate r, and for tail count k let $A_{t,k}(r)$ be the mean of the k largest $\Delta_{t,j}(r)$ (so $k = H$ is the signed mean and $k = 1$ is the maximum). Define the conservative suffix-safe rate $r^*_{t,k}(\epsilon) = \min\{r \in \mathcal{R} : A_{t,k}(r') \le \epsilon \ \forall r' \ge r\}$.

Proposition 2 (risk-capacity ordering). If $k_1 \ge k_2$ then $A_{t,k_1}(r) \le A_{t,k_2}(r)$ for every r, hence $r^*_{t,k_1}(\epsilon) \le r^*_{t,k_2}(\epsilon)$. *Proof.* The mean of the top-k elements of a fixed vector cannot decrease as k shrinks, so the suffix-feasible rank set for k_2 is a subset of that for k_1; their minima are ordered. This holds for any realized loss vector, including the empirically nonmonotone rank-loss curves we observe, and requires no assumption about the sign or shape of $\Delta_{t,j}$. It is what licenses reporting an ordered risk-capacity spectrum (Section 5.1) rather than only two endpoints.

Under a bounded-Lipschitz assumption on the future logit map, nested-prefix truncation additionally admits the sensitivity envelope $|\Delta_{t,j}(r)| \le \sqrt{2} K_{t,j} \|e_t(r)\|_2$, where $e_t(r)$ is the discarded latent tail and $K_{t,j}$ a local Lipschitz constant; this gives a monotone safety bound consistent with, but not proving, the realized risk-capacity ordering. Full derivations, the constrained joint rate-distortion formulation that motivates the router objective, a formal statement of the straight-through surrogate's scope, and further falsifiable predictions are given in `notes/theory_contextual_tail_rate.md`.

3.3 Corrected future-loss capacity analysis

For each source position p and candidate rank r, we simultaneously zero channels outside the layer-specific nested prefix at position p in every layer; all other positions remain full rank. Let $\delta_{p,j}(r)$ denote the resulting change in next-token cross-entropy at future-loss offset $j \in \{0, \ldots, H-1\}$. We define

$$A_{p,\text{mean}}(r) = \frac{1}{H} \sum_{j=0}^{H-1} \delta_{p,j}(r), \qquad A_{p,\max}(r) = \max_{0 \le j < H} \delta_{p,j}(r),$$

and, for $A \in \{\text{mean}, \max\}$,

$$r^*_{p,A} = \min\{r \in \mathcal{R} : A_p(r') \le \epsilon \text{ for every } r' \in \mathcal{R},\ r' \ge r\}.$$

The suffix condition guards against nonmonotone rank-loss curves. We report the mean of $r^*_{p,A}$ over 768 probed positions, separately for the mean- and maximum-over-horizon criteria, with $H = 32$ and $\epsilon = 0.10$ nat. Version 4 fixes an earlier off-by-one error by including `logits[p]`, the first prediction affected by masking source position p. Calibration repeats, calibration windows, and evaluation windows are nonoverlapping. Each scale uses 24 sequences and 32 probed positions per sequence, with checkpoint, data, source, and record hashes stored for audit.

3.4 Contextual router

Layer 0 runs at full rank. Let z_t be the normalized pre-attention state entering layer 1 after the layer-0 contextual update. A two-layer MLP router produces tier logits

$$a_t = g_\theta(z_t), \qquad r_t \in \mathcal{T}.$$

Model	Parameters	Layers	d_{model}	Heads	d_c	d_R	Context	Training tokens seen
MLA-30M	30.6M	6	384	6	256	32	256	24.576M sampled tokens
MLA-122M	122.14M	12	768	12	384	32	384	147.46M tokens
MLA-250M	249.27M	16	1024	16	512	32	384	55.3M sampled tokens

The selected tier is shared by all downstream layers. The 30M and 250M-coarse policies use a 4-tier grid at 6.25%/25%/62.5%/100% of d_c ($\{16, 64, 160, 256\}$ at 30M, $\{32, 128, 320, 512\}$ at 250M). The 122M and 250M-fine policies use an 18-point grid from 16 to d_c with 16-step spacing up to 64 and 32-step spacing thereafter (14 tiers at 122M's $d_c = 384$, 18 tiers at 250M's $d_c = 512$).

3.5 Joint-rollout training

Deployment uses the hard one-hot choice $y_t = \text{onehot}(\arg\max a_t)$. During training we use the straight-through estimator

$$\hat{y}_t = y_t + p_t - \text{stopgrad}(p_t), \quad p_t = \text{softmax}(a_t/\tau).$$

The forward pass therefore matches hard-tier inference, while gradients follow the soft probabilities. The frozen base language model is optimized only through the router objective

$$\mathcal{J}(\theta) = \mathcal{L}_{LM}(\hat{y}) + \lambda \frac{1}{T d_c} \sum_t \sum_k p_{t,k} \mathcal{T}_k.$$

The base model receives no gradients. Candidate λ values are trained on 16 sequences and selected on four validation sequences. The selection rule chooses the minimum-byte candidate within a +0.15-nat validation loss budget that also beats an exact-byte static control. The chosen policies are $\lambda = 0.4$ at 30M, $\lambda = 0.8$ at 122M, $\lambda = 0.4$ at 250M-coarse, and $\lambda = 0.8$ at 250M-fine; the two 250M policies are trained and frozen independently to disentangle tier granularity from scale (Section 5.2). The 122M and 250M-fine routers use random router initialization; the others initialize from the isolated-position supervised router. Four older development sequences were repeatedly observed during method development and are treated only as exploratory; they are not used for any confirmatory claim.

3.6 Exact-byte controls

For the primary static control, let the router's downstream rank sum on one sequence be R. Writing $R = qT + m$, the control assigns rank $q + 1$ to m random positions independently of token content and rank q to all others. Twenty allocations are averaged. It has exactly the same downstream rank sum and cache tensor-payload bytes as the router, while approximating a continuous uniform rank. This placement is position-independent only after borrowing the router's content-dependent realized budget for that sequence; it is not a globally fixed-budget policy.

The secondary control permutes the router's exact tier histogram across positions 20 times. It preserves both the cache tensor-payload byte budget and the marginal tier distribution, isolating whether contextual rank placement matters.

4 Experimental Setup

4.1 Models and data

We train three decoder-only MLA language models on TinyStories-tokenized data, spanning an 8x parameter range.

The 30M model was trained for 3,000 steps from an approximately 8.8M-token training corpus and its checkpoint has validation loss 1.9618. The 122M and 250M models share byte-identical held-out validation streams (same TinyStories corpus construction, seed, and split fraction) despite their different architectures. The 122M final step-8,000 checkpoint has validation loss 1.5662; the best logged value, 1.5179 at step 7,250, was not checkpointed. The 250M model was trained for 3,000 steps of 48 sequences of 384 tokens (55.3M tokens, about 0.63 passes over the 88M-token training stream) and reached validation loss 1.6588 — fewer tokens than the 122M model and a higher validation loss. The three models therefore differ in training budget as well as size, and every cross-scale comparison in this paper confounds the two. We make no best-checkpoint claim for any model: all three use the final training step. 30M was trained locally (Apple Silicon MPS); 122M and 250M were trained on Kaggle (Tesla P100). The risk-capacity analysis is additionally replicated on the public DeepSeek-V2-Lite checkpoint ($d_c = 512$, 27 layers) on WikiText-103, without training anything (Section 5.1).

4.2 Confirmation protocol

For 30M and 122M, commit `a4bcc7f` froze, before any fresh-window result was produced, each policy file, its SHA-256 hash, the deterministic sampling seed (91,827), 24 fresh sequences, 20 control permutations, the primary endpoint, and the success rule, in `notes/fresh_confirmation_protocol.md` together with the policy files themselves. The machine- readable `experiments/fresh_confirmation_manifest.json` that later scripts authenticate against was written afterwards, in the same commit as the 30M/122M results (`080af82`); it restates the frozen values but was not itself committed first, and the oracle and contextual-summary hashes it adds were not part of the 30M/122M freeze. The 250M-coarse and 250M-fine entries were each committed to that manifest before their confirmation runs (`1ee1528` and `010e670`). The 250M-fine entry was added after the 250M-coarse result was known, to test the tier-granularity explanation, and its run reused the same 24 fresh windows: both 250M protocols use seed 91,827 and the same exclusion set, so the 250M-fine confirmation is not on untouched data. Its policy was trained and selected on the original training/validation sequences only, so the reuse cannot have tuned the policy, but it is a second look at windows whose coarse-grid outcomes had been seen. Fresh windows are sampled from the held-out evaluation region and separated from all 24 prior oracle/development windows and from one another by at least `block_size + 1` tokens. The primary endpoint is paired per-sequence router loss minus mean exact-byte static loss. A configuration passes when the upper bound of its 95% sequence-cluster bootstrap interval is below zero. We also report exact one-sided paired sign-flip tests and sequence win counts. `experiments/audit_fresh_confirmation.py` reconstructs the seeded sampler and recomputes every statistic from the stored per-sequence rows before a result is treated as valid; it checks the arithmetic, sampling and byte equality but does not rerun the model, so the per-sequence losses themselves are not recomputed by it. `experiments/audit_joint_training_replay.py` compares a separately retrained policy file against the frozen one and confirmed a bit-for-bit match; the retrained files were not committed, so that check cannot be repeated from the repository alone without retraining.

5 Results

5.1 The full risk-capacity spectrum is monotone and nearly scale-invariant across 30M-250M

We extend the two-point mean/max analysis to a full upper-tail spectrum computed from the same corrected, provenance-tracked windows, now at **three** scales spanning an 8x parameter range: for each source position we retain the largest k of the $H = 32$ future loss deltas, for $k \in \{32, 16, 8, 4, 2, 1\}$, and take the smallest suffix-safe rank.

Every step is monotone nondecreasing at all three scales, exactly as Proposition 2 (risk-capacity ordering) guarantees. The normalized **tail-capacity premium** $\mathrm{TCP} = \mathbb{E}[r^*_{\max} - r^*_{\mathrm{mean}}]/d_c$ is 0.6507 (95% CI [0.6326, 0.6669]) at 30M, 0.6470 (CI [0.6283, 0.6652]) at 122M, and 0.6464 (CI [0.6257, 0.6665]) at 250M — three overlapping intervals across an 8x parameter range, making this the most scale-consistent quantitative result in the paper. The rank grid is 16-64 in steps of 16 and then steps of 32 up to d_c at every scale; an earlier run at 250M stopped the grid at 384 and jumped to 512, which rounded every requirement in (384, 512]

k (of 32)	$\alpha = 1 - k/H$	30M $/d_c$	122M $/d_c$	250M $/d_c$
32 (mean)	0.000	9.00%	8.22%	7.15%
16	0.500	21.47%	23.00%	22.12%
8	0.750	39.65%	39.52%	38.60%
4	0.875	55.75%	54.22%	54.16%
2	0.938	66.94%	65.56%	64.95%
1 (max)	0.969	74.07%	72.92%	71.78%

up to full rank and overstated the 250M tail rates by up to 4.3 points (the max-criterion rate was reported as 76.03% and the premium as 0.689). All 250M numbers here are from the corrected rerun with the same seeds, checkpoint, windows and channel orders.

The separation is driven by pervasive cancellation, not rare catastrophic tokens, at all three scales. At the rank that is safe under the signed mean criterion, the positive-part mean loss (mean of $\max(\Delta, 0)$ over the horizon) is 2.05x (30M), 2.31x (122M), and 2.15x (250M) larger than the signed mean, and 93.1%/94.1%/96.1% of such "mean-safe" positions still have at least one future offset whose loss increase exceeds ϵ. If harm were concentrated in a few rare spikes, the vast majority of mean-safe positions would have zero offsets above ϵ; instead nearly all of them do at every scale we tested, and the signed mean is kept small by cancellation against negative excursions elsewhere in the horizon. **Figure 2** (`figures/elasticmla_risk_spectrum.pdf`) shows the spectrum and this diagnostic across all three scales.

The 250M model (Section 4.1) contributes to the risk-capacity spectrum result here and, as described in Sections 5.2-5.4, to a full contextual-router replication under two different tier grids.

The normalized 122M-minus-30M difference in the original mean/max endpoints is -0.781 percentage points for the mean (95% bootstrap CI: [-1.617, +0.076]) and -1.156 points for the maximum (CI: [-3.849, +1.611]); both include zero. This is not an equivalence test and does not establish scale invariance of the endpoints, but the tail-capacity premium result above is a stronger and separate scale-consistency finding computed across the whole spectrum and now three scales rather than two endpoints at two scales.

For 30M and 122M the per-record raw deltas underlying this spectrum were computed on ephemeral cloud job storage and are not independently re-auditable; only the aggregate summary (with verified checkpoint/data provenance) was recovered. The corrected 250M rerun kept its raw records, which the summary authenticates by hash. See `notes/risk_capacity_spectrum_results.md`.

These rates are basis-dependent, and a large part of the tail requirement is the cost of axis alignment. Every rank above is measured in the model's own latent coordinates, ordered by saliency — a permutation, the most restricted nested basis available. Rerunning the identical truncation probe in a rotated basis cuts the worst-offset requirement to 23.56%/23.25%/21.89% at 30M/122M/250M, a paired fall of about 50 percentage points at every scale (49.7-50.5) with bootstrap intervals nowhere near zero (Section 6, `notes/basis_dependence_prediction7.md`). We report the coordinate-basis numbers because they are what the deployed packed cache actually uses and what the router was trained against, but in these models they should be read as properties of *this basis*, not as an intrinsic rate of the latent code. On a production-trained MLA model the picture is different (next paragraph).

The spectrum replicates on a public, production-trained MLA model, and the basis effect largely does not. Everything above was measured on models trained inside this project on TinyStories. DeepSeek-V2-Lite [2] (27 layers, 15.7B parameters with 2.4B active, trained by its authors on 5.7T tokens) caches exactly the object this paper is about: a 512-wide latent, the same d_c as our 250M model, plus a 64-wide decoupled RoPE key. We applied the identical probe to it (`experiments/`

k (of 32)	250M coord	V2-Lite coord	V2-Lite pca	V2-Lite grad	V2-Lite model metric
32 (mean)	7.15%	8.22%	5.06%	5.94%	5.19%
16	22.12%	22.92%	12.72%	16.58%	13.79%
8	38.60%	40.14%	25.68%	31.18%	26.68%
4	54.16%	56.86%	42.36%	49.51%	43.47%
2	64.95%	70.69%	56.90%	64.97%	57.44%
1 (max)	71.78%	78.65%	67.34%	73.47%	67.58%
TCP	0.646	0.704	0.623	0.675	0.624

Scale (tiers)	Mean rank	Packed / fixed MLA	Δloss vs full	Router - static	95% CI	Wins	Exact p	Result
30M (4 coarse)	145.78 / 256	68.80%	+0.1001	**-0.01959**	[-0.02911, -0.00938]	20/24	0.000655	success
122M (14 fine)	206.93 / 384	61.46%	+0.1823	**-0.03251**	[-0.04575, -0.01958]	21/24	0.0000493	success
250M (4 coarse)	317.15 / 512	66.82%	+0.1158	+0.00190	[-0.00222, +0.00610]	13/24	0.806	**failure**
250M (18 fine)	278.67 / 512	60.19%	+0.1659	**-0.01165**	[-0.01911, -0.00394]	17/24	0.00438	success

`analyze_public_mla_spectrum.py`): the cached latent at a source position is truncated at every layer at once through a forward hook on the normalization whose output the cache stores, the RoPE key is left intact as in our models, and r^* uses the same suffix rule, $H = 32$, $\epsilon = 0.10$ nat, rank grid and four bases, calibrated on 32 training sequences. Because TinyStories is out of distribution for this model, text is WikiText-103: 24 test sequences of 512 tokens after a BOS token that is never probed, 16 positions each (384 probes, half our 768). The weights run in float16 on two T4 GPUs; the full-rank probe of every basis reproduces the unmodified loss exactly (maximum deviation 0.0), and the unmodified mean token loss is 2.14 nats.

In the coordinate basis the production model's spectrum is monotone and close to ours at every tail count, slightly heavier at the far tail: its premium, 0.704 (95% CI [0.676, 0.731]), is above the 0.646-0.651 of our three models and its intervals do not overlap theirs, so we claim the same shape and magnitude, not the same value. The cancellation signature replicates: at the mean-safe rank the positive-part mean loss is 2.10x the signed mean, and 96.6% of mean-safe probes still have an offset above ϵ (2.05-2.31x and 93.1-96.1% in our models). The basis ablation does not replicate in magnitude. Rotation still helps — every rotated basis lowers the requirement at every tail count, with paired per-probe intervals excluding zero — and the bases still rank as in our models (the model-metric frame ties principal directions at the worst offset, CI [-1.3, +1.9], and ordering by first-order loss effect is worse than both at every tail count), but the worst-offset fall is 11.3 points for principal directions (paired CI [9.1, 13.5]) and 11.1 for the model-metric frame, against about 50 in our models; 59 of 384 probes still need the entire 512-wide latent under principal directions (106 under coordinates). In a production-trained MLA model the tail requirement is therefore mostly a property of the code, and the coordinate-basis spectrum is a closer description of it than our own models' basis ablation suggested. Two qualifications: 275 of 384 coordinate-basis probes have their mean-criterion requirement at the grid's smallest rank (16, 3.1% of d_c), so the 8.22% mean is an upper bound and the premium a lower bound; and the four bases were measured in two runs, which share bit-identical evaluation tokens, probe positions and baseline losses (checked before pairing, `experiments/combine_public_mla_runs.py`) but each computed its own calibration statistics, whose float16 backward is not bit-deterministic on GPU. See `notes/public_mla_deepseek_v2_lite.md`.

5.2 Fresh contextual routing beats exact-byte static allocation, with a tier-granularity dependence at 250M

At 30M and 122M the router satisfies its separately pre-specified primary criterion. At 250M with the *same* coarse 4-tier grid used at 30M, it does not: the point estimate is in the wrong direction and the CI includes zero. Because 30M used coarse tiers and 122M used a fine 14-tier grid, this coarse-only 250M result confounds scale with tier granularity, so we ran a second, separately pre-registered confirmation at 250M with an 18-tier fine grid (mirroring 122M's scheme; same base checkpoint, oracle, and channel orders; random router initialization, exactly as the 122M fine-grid policy was built). It **succeeds**: -0.01165 nat, CI excludes zero, p=0.0044. This shows the initial 250M failure was, at least in part, a tier-resolution artifact of our own

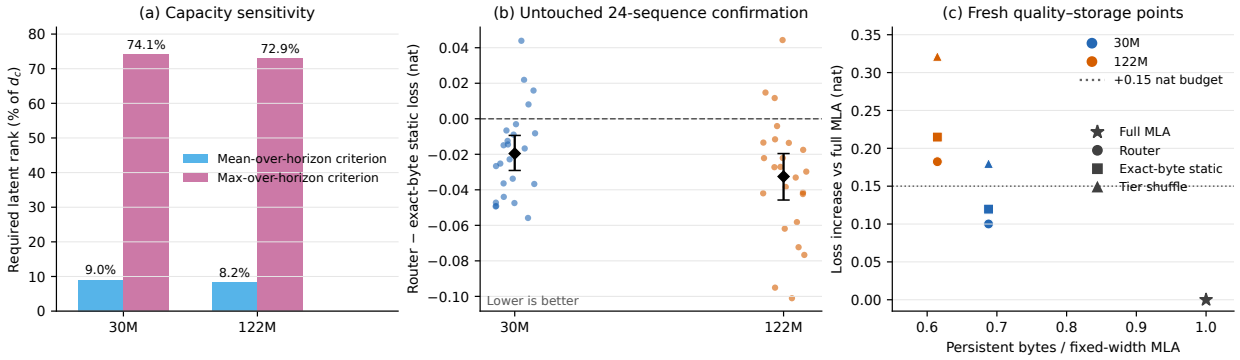


Figure 1: (a) Mean required rank under the mean- and maximum-over-32-token-horizon criteria, averaged over 768 probed positions per scale. (b) Per-sequence router-minus-control differences; dots are sequences, diamonds are means, and error bars are paired sequence-bootstrap 95% intervals. (c) Mean fresh-sequence loss increases versus full MLA; both controls have exactly the router's per-sequence cache tensor-payload byte count. Points are discrete evaluated configurations, not an interpolated operating curve. Panel (a) shows only the mean/max endpoints; see Figure 2 for the full six-point risk-capacity spectrum. Panels (b)-(c) show only 30M/122M; see Section 5.2's table and `notes/contextual_router_250m_tier_granularity_diagnostic.md` for both 250M configurations.

design rather than purely an intrinsic scale limitation: at $d_c = 512$, four coarse tiers are too low-resolution for the router to reliably beat random matched-budget allocation, but 18 finer tiers restore that advantage. We did not adjust either protocol after seeing its own result; both were frozen in advance (`experiments/fresh_confirmation_manifest.json`, scale keys `250m` and `250m_fine`), and we report both rather than only the more favorable one. The 250M-fine protocol was, however, designed after the 250M-coarse outcome and evaluated on the same 24 windows (Section 4.2), so it is weaker evidence than a confirmation on new windows would be. The tier-grid choice at 122M has a similar history: the coarse four-tier grid there produced no candidate meeting the validation budget, and the 14-tier fine grid was adopted in its place before any fresh-window run — which is itself consistent with the tier-granularity explanation, but means the 30M-coarse/122M-fine contrast was not designed in advance. Neither protocol pre-specified a family-wise multi-scale test, so we do not elevate these results to a single combined confirmatory claim; each frozen protocol stands on its own. Full detail: `notes/contextual_router_250m_tier_granularity_diagnostic.md`.

The exact-histogram shuffle comparison is more forgiving at every scale/tier configuration we tested: router-minus-shuffle is -0.07989 nat at 30M (CI: [-0.08999, -0.06956]), -0.13907 nat at 122M (CI: [-0.15439, -0.12408]), -0.01765 nat at 250M-coarse (CI: [-0.02306,-0.01228]), and -0.03551 nat at 250M-fine (CI: [-0.04404,-0.02621]), with the router winning the large majority of sequences at every configuration. So within-sequence placement of the router's own chosen tier multiset consistently helps, even at the one configuration (250M coarse) where the overall rate selection does not beat random allocation.

5.3 Quality-constraint generalization is imperfect, and is independent of the allocation result

The 30M and 250M-coarse policies remain within the +0.15-nat selection budget on fresh sequences (fresh Δloss 0.1001 and 0.1158). The 122M and 250M-fine policies do not: validation Δloss was +0.1408 (122M) and +0.1483 (250M-fine), while fresh Δloss is +0.1823 (122M) and +0.1659 (250M-fine). Quality-budget calibration and allocation efficiency are separate axes and do not track each other: 122M keeps its allocation advantage over static despite missing its budget; 250M-coarse stays within budget yet loses its allocation advantage; 250M-fine regains the allocation advantage but also misses its budget. A deployable system should use a larger calibration set, a conservative risk margin, or online fallback to a higher tier for the quality-budget issue, and a fundamentally stronger training signal (Section 5.4, Section 6) for the allocation-advantage issue.

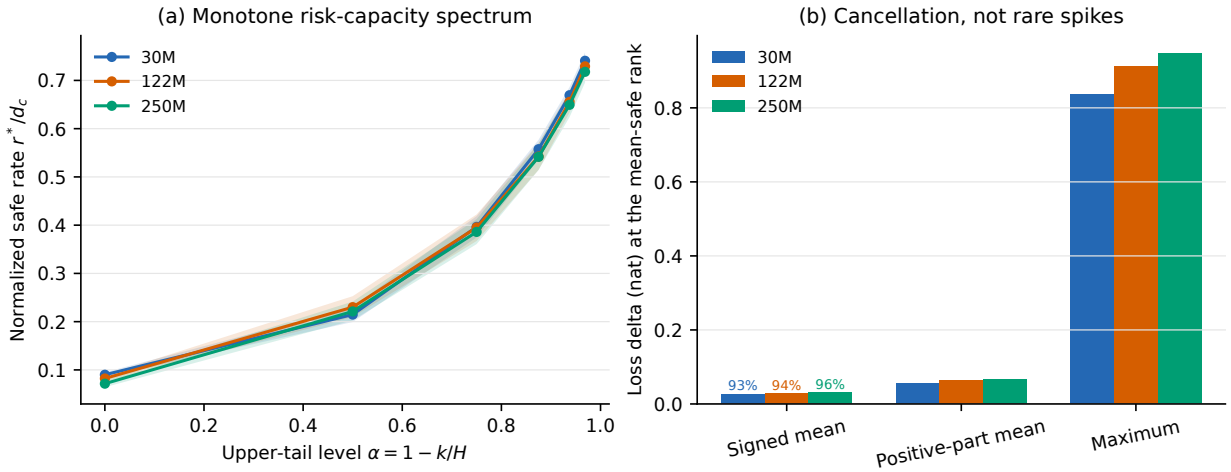


Figure 2: (a) Normalized suffix-safe rate against upper-tail level $\alpha = 1 - k/H$ at all three scales, with paired sequence-bootstrap 95% bands; monotonicity is guaranteed by Proposition 2. (b) Signed mean, positive-part mean, and maximum loss delta at the rank that is safe under the signed mean criterion, annotated with the fraction of records having at least one future offset exceeding ϵ; the gap between signed and positive-part means shows cancellation rather than rare spikes.

Scale (tiers)	Control	Router - control, own histogram (nat)	95% CI	Result	Router - control, router's histogram (nat)	Result
30M (coarse)	position	-0.0225	[-0.0326, -0.0117]	**router wins**	-0.0804	**router wins**
30M (coarse)	lexical identity	+0.0024	[-0.0079, +0.0130]	tie	-0.0049	tie
30M (coarse)	token rarity	-0.0084	[-0.0162, +0.0002]	tie	-0.0437	**router wins**
30M (coarse)	token type	-0.0146	[-0.0241, -0.0050]	**router wins**	-0.0689	**router wins**
122M (fine)	position	-0.0359	[-0.0497, -0.0227]	**router wins**	-0.1309	**router wins**
122M (fine)	lexical identity	-0.0033	[-0.0133, +0.0065]	tie	-0.0160	**router wins**
122M (fine)	token rarity	-0.0045	[-0.0175, +0.0091]	tie	-0.0418	**router wins**
122M (fine)	token type	-0.0284	[-0.0421, -0.0151]	**router wins**	-0.1139	**router wins**
250M (coarse)	position	+0.0012	[-0.0031, +0.0054]	tie	-0.0210	**router wins**
250M (coarse)	lexical identity	+0.0045	[+0.0012, +0.0080]	router loses	-0.0019	tie
250M (coarse)	token rarity	+0.0071	[+0.0028, +0.0114]	router loses	-0.0142	**router wins**
250M (coarse)	token type	+0.0009	[-0.0035, +0.0053]	tie	-0.0217	**router wins**
250M (fine)	position	-0.0133	[-0.0208, -0.0050]	**router wins**	-0.0397	**router wins**
250M (fine)	lexical identity	+0.0022	[-0.0053, +0.0100]	tie	-0.0103	**router wins**
250M (fine)	token rarity	-0.0001	[-0.0084, +0.0090]	tie	-0.0164	**router wins**
250M (fine)	token type	-0.0114	[-0.0189, -0.0039]	**router wins**	-0.0430	**router wins**

5.4 At equal bytes the router beats position and type heuristics, and ties token identity

The random and matched-histogram-shuffle controls above are content-independent given the router's realized budget, so they are weak baselines: a simple, cheap, strictly causal rule (depending only on the current token and/or absolute position, never on future tokens) might place the same bytes as well. We fit four such rules — absolute position, token identity (frequency-smoothed), inverse token rarity, and coarse token type — as predictors of the router's own tier, using only the original 16 training sequences, and evaluate them once on the same 24 frozen fresh sequences. These are post-confirmation, exploratory comparisons on already-opened windows.

Each rule must spend exactly the router's bytes on every sequence, and we hold it to that in two ways. In the stricter comparison for the router (*rule picks its own histogram*), the rule receives only the router's per-sequence total downstream rank and distributes it by its own score: $r_t = \lfloor s_t + b \rfloor$ clipped to $[1, d_c]$, with b solved per sequence and the integer remainder given to the largest fractional parts, so that — like the exact-byte static control — it is not restricted to the router's tiers. In the second (*router's histogram*), the rule receives the router's realized tier multiset and only decides which positions get which tier (highest score, highest tier; score ties broken at random and averaged over five draws), the same frame as the shuffle control. Every evaluation asserts byte equality with the router.

Scale	Cache bytes (router / full)	Peak allocated (router / full)	Stage 1: as first written	Stage 2: vectorized	Stage 3: + in-place append
30M	67.4%	99.7%	1412.6 ms (**167.9x**)	25.48 ms (**3.00x**)	18.83 ms (**2.42x**)
122M	59.6%	99.3%	3592.9 ms (**201.1x**)	62.13 ms (**2.96x**)	42.11 ms (**2.37x**)

When each rule chooses its own histogram, the router wins 6 of 16 comparisons, ties 8 and loses 2 (CI excluding zero in each direction). The pattern is consistent across scales: the router beats absolute position and coarse token type at every configuration except 250M-coarse, and it never beats token identity — lexical identity ties it at 30M, 122M and 250M-fine and beats it at 250M-coarse. Both losses are at 250M-coarse, the configuration whose four-tier grid also fails the primary confirmation (Section 5.2). Restricted to the router's own tier multiset, the rules lose 14 of 16 and tie 2, but that comparison handicaps them: the router's histogram was chosen together with its placement, and a near-uniform histogram is itself better than a shuffled router histogram (Section 5.2). We therefore treat the own-histogram column as the operative comparison.

The supported reading is narrower than "contextual routing works". At equal bytes, the router's placement is better than position- and type-based placement but statistically indistinguishable from a frequency-smoothed per-token lookup table fitted to its own decisions on 16 sequences. Much of what the router learned is therefore expressible as token identity; we have not shown that contextual information beyond the current token improves placement. An earlier version of this comparison let each rule run at its own validation-selected bytes, which differed from the router's by -6.5% to +13.0% per configuration and decided almost every outcome by budget rather than placement; it is superseded by the byte-matched results above (`experiments/evaluate_causal_heuristic_routers.py -placement router-bytes|router-histogram`, `experiments/contextual_router_*/causal_heuristic_controls_router_bytes,histogram*.json`).

5.5 Measured GPU memory and latency: an initial severe cost, and two subsequent fixes

Section 3.1's byte formula is a derived tensor-payload count. We benchmarked real prefill-then-128-step incremental decode on a Tesla T4 GPU at both scales, comparing full-width dense MLA, a uniform packed cache matched to the router's realized rank, and the router's packed cache (`experiments/benchmark_cache_memory_latency.py`).

Multipliers are against full-width dense MLA measured in the same run (8.41 ms and 17.87-20.99 ms per decode step at 30M and 122M for stages 1-2; 7.79 ms and 17.74 ms in the stage-3 paired run).

Measured cache-byte ratios closely track the derived formula and are unaffected by the fix below. Peak allocated memory is essentially unchanged: 99.7% (30M) and 99.3% (122M) of full MLA for the router, 99.0% and 98.4% for the matched uniform cache, identical across three repeats. Peak is dominated by the weights and the prefill's vocabulary-sized logits, not by the cache. An earlier version of this benchmark reported a 3.5-4.2% router reduction; it came from the harness, whose full and uniform paths kept their prefill logits referenced into decoding while the router path released them. With every path dropping its prefill outputs, a rerun of the same configurations (`experiments/contextual_router_30m,122m/measured_cache_memory_latency_review_rerun.json`) gives the column above and per-step regressions of 2.23-2.42x and 2.40-2.41x, consistent with stage 3. Decode latency was initially two orders of magnitude worse for the packed path, traced to three independent unvectorized Python loop sites: `pack_latents/unpack_latents/append_packed_latents` (`code/elastic_mla/elastic_cache.py`) reconstructed the entire cached history with a per-token Python loop on every decode step, and `MultiHeadLatentAttention.forward_cached_packed` (`code/elastic_mla/mla.py`) separately built its per-token rank mask with a nested `for b /for t` Python double loop. We rewrote all three as vectorized PyTorch operations (`torch.repeat_interleave` plus a single gather for pack/unpack; a broadcasted inverse-permutation comparison for the rank mask), changing nothing about the packed representation, byte accounting, or numerical results — verified by 500 random-trial cross-checks against the original implementations (`torch.equal`, zero mismatches) and the full unit suite (49/49, both before and after). This cuts the packed decode step by 55-58x at both scales, reducing the latency regression from 168-201x to roughly 3x (stage 2).

The stage-2 residual had two named sources: the dense-reconstruction gather before every attention call, and `append_packed_latents` unpacking and repacking the *entire* history on every single-token decode step rather than appending the new token in place. We then removed the second one (stage 3). The obstacle was the flat buffer's token ordering: values were stored row-major `(batch, sequence)`, so a new token for batch row 0 landed in the middle of the buffer and shifted every later row's offsets, making a full rebuild unavoidable. Reordering the flat buffer sequence-major `(sequence, batch)` makes every new token sort after every stored token, so appending becomes a concatenation of the packed buffers. Profiling the result showed that the repacking had in fact not been the dominant cost at all: the rewritten append still spent roughly 200x the time of its own data movement (1.91 ms against 0.007 ms for the value concatenation, one 30M layer at B=8, S=320) inside the defensive validation guards, which re-derived a permutation check, two bounds reductions and several device-to-host offset reads on every decode step for every layer. Every guard is retained, but each now runs where its guarantee is established — full permutation validation on the prefill call that creates the cache's coordinate system, equality against the cache's own validated order on each decode step — and the bounds and offset audits are fused into single comparisons. Neither change alters the packed representation, the byte accounting, or the computation: verified by a 500 random-trial cross-check against the previous implementation, a 54/54 unit suite, and an end-to-end 30M router decode producing bit-identical logits (maximum absolute difference 0.0), identical chosen ranks and identical cache bytes.

Stage 3 was measured as a *paired* benchmark: the same unmodified benchmark script run alternately against the stage-2 and stage-3 sources, three repeats each, inside a single T4 job so both share one GPU and thermal state. In that run the stage-2 code re-measured at 3.09x (30M) and 2.99x (122M), reproducing the separately measured stage-2 column above, while the untouched dense control moved by 1% or less; stage 3 measures 2.42x and 2.37x, a 1.26-1.29x per-step improvement. The remaining cost is now dominated by the first source, the dense reconstruction before each attention call, which is inherent to a design whose persistent representation is packed: removing it means either keeping a dense working set resident (which would make the persistent-byte saving irrelevant at decode time, a change of claim rather than an optimization) or computing the per-head key/value projections directly against the ragged packed buffer. See `notes/measured_cache_memory_latency.md` for full results at both configurations and both scales.

6 Validity, Reproducibility, and Limitations

Statistical scope. The unit of inference is a full sequence, not an individual token. Twenty- four clusters per scale support paired uncertainty estimates but remain modest. The exact sign-flip p-values enumerate all sign assignments conditional on the observed magnitudes, but their inferential interpretation still requires exchangeability of paired-difference signs under the null. The intervals and tests do not incorporate checkpoint, policy-seed, or training-run variability. We evaluate one checkpoint per scale, one data distribution, and one router seed, and the three checkpoints also differ in training tokens (Section 4.1). The one external replication (DeepSeek-V2-Lite, Section 5.1) covers the risk-capacity spectrum and the basis ablation only, on one public checkpoint, one text distribution (WikiText-103) and 384 probes from 24 sequences; the router was not trained or evaluated on it. At 30M and 122M the per-record raw deltas underlying the risk-capacity spectrum (Section 5.1) were computed on ephemeral cloud job storage and could not be retrieved after job completion; only the aggregate summary statistics (independently checkpoint/data-hash verified) were recovered, so those two spectra are not re-auditable at the per-position level the way the confirmation results are.

Threshold scope. Raw rank-loss curves are frequently nonmonotone: 83.6%/89.3%/92.1% of mean-over-horizon curves and 59.4%/69.7%/78.0% of maximum-over-horizon curves at 30M/122M/250M (any rise counts; the grid has 10/14/18 points, so these frequencies are not comparable across scales). Accordingly, r^* uses the conservative suffix rule and is defined only on the tested discrete rank grid; it should not be interpreted as a smooth or uniquely identified intrinsic rank.

Control scope. The primary control randomizes rank placement conditional on each router sequence's realized total rank; it is therefore position-independent, but not a globally fixed- budget policy. Twenty random allocations estimate its per-sequence loss. Section 5.4 adds four causal heuristic baselines (position,

lexical identity, rarity, type) held to exactly the router's per-sequence bytes across all four scale/tier-grid configurations (16 scale-heuristic comparisons): the router wins 6, ties 8 and loses 2. Those comparisons reuse the opened fresh windows and are exploratory. We still lack a comparison against separately optimized global-budget static policies, a router and packed cache trained and evaluated in a rotated latent basis (Section 6, Basis scope, measures only the probe), and eviction/quantization baselines from the wider KV-cache compression literature.

Oracle scope (now measured). Capacity labels are isolated-position interventions, not joint-rollout-optimal labels. Joint training alleviates but does not solve this limitation. We quantified the gap with the second-order mixed difference $L_{\{t_1,t_2\}} - L_{\{t_1\}} - L_{\{t_2\}} + L_{\emptyset}$, which vanishes identically for any additive distortion, over 1008 position pairs per scale. The residuals are nonzero — so the additive special case of Section 3 is falsified and isolated labels are indeed structurally misaligned with joint routing — but the misalignment is local: as a fraction of the individual effects it is 9.09%/12.17%/13.17% at 30M/122M/250M for adjacent positions averaged over the three tested ranks (16.79%/23.28%/22.78% at the heaviest truncation, rank 16, alone), decays monotonically with separation, and falls below 1.3% beyond a separation of 32 tokens. It grows with truncation severity (averaged over separations, 5.69% at rank 16 to 1.07% at rank 224 at 30M). We do not draw a scale trend from it: the tested ranks are fixed absolute values, so they are a different fraction of d_c at each scale (the middle rank is 50%/50%/38% of d_c), and at the one rank shared by all three, rank 16, the 250M fraction (7.98%) is below the 122M one (8.41%). With 144 pairs per cell from eight sequences and no interval, the data neither establish nor exclude a growth with scale. See `notes/pair_interaction_prediction5.md`.

System scope (now measured, not only derived). We benchmarked real prefill+128-step incremental decode on a Tesla T4 GPU at both scales (`experiments/benchmark_cache_memory_latency.py`, `notes/measured_cache_memory_latency.md`). Measured persistent cache bytes closely track the byte-formula predictions (67.2-67.4% of full MLA at 30M, 54.2-59.6% at 122M). Peak allocated GPU memory is essentially unchanged (0.3-1.6% lower across packed configurations), because the cache is a small part of peak at these model sizes and context lengths. Decode in this benchmark runs to positions 320 (30M, trained context 256) and 384 (122M), so the 30M decode steps extend past the training context. Decode latency was initially 168-201x slower for the packed path at both scales, traced to three unvectorized Python loop sites; we rewrote all three as vectorized PyTorch operations, reducing the regression to roughly 3x, and then made the cache append in place rather than repack its whole history each step, reducing it to 2.4x (Section 5.5), validated against random-trial correctness cross-checks, the full unit suite, and bit-identical end-to-end decode output. We still do not claim latency parity, throughput gains, or superiority to optimized MHA/GQA/FlashMLA kernels — a 2.4x per-step cost remains, now dominated by the packed path's dense-reconstruction gather before each attention call — but this is no longer a two-orders-of-magnitude blocker, and the remaining source is a consequence of keeping the persistent representation packed rather than an unaddressed implementation defect.

Quality scope. Every compressed policy increases loss relative to full MLA, and the 122M and 250M-fine policies miss their held-out +0.15-nat budget (30M and 250M-coarse do not). Task-level generation quality is not evaluated.

Architecture scope. Layer 0 is always full, and one tier is shared by every downstream layer. Per-layer routing could improve efficiency but increases metadata, training complexity, and kernel divergence.

Basis scope (a negative result for the reported rates). The nested code is a *permutation* of the model's latent coordinates, ordered by a gradient-times-activation saliency. We tested whether that restriction inflates the reported rates by rerunning the same truncation probe in two rotated orthonormal bases, calibrated on the same tokens: principal directions of $E[cc^\top]$, and eigenvectors of $\mathrm{sym}(E[gc^\top])$, whose quadratic form is the mean first-order loss effect of dropping a direction (its diagonal in the coordinate basis is the signed version of the saliency we use, so the published ordering is the axis-aligned special case of the same criterion). The rotation is applied to the weights rather than the probe — with W_{DKV}, W_{UK}, W_{UV} bias-free, substituting $U^\top W_{DKV}$ and $W_{UK}U$, $W_{UV}U$ leaves the full-rank function unchanged while making a prefix mask a projection onto $\mathrm{span}(U_{:,:k})$, which is verified at full rank to at most 3.1×10^{-5} logit deviation.

Because every basis is measured on the same probes, we compare them by paired per-probe differences rather than by the overlap of marginal intervals.

At all three scales the worst-offset required rate falls by about 50 percentage points (paired differences 50.5/49.7/49.9), to 23.56%/23.25%/21.89% at 30M/122M/250M, and the required-rank histograms shift accordingly: under the coordinate basis at 30M, 410 of 768 probes need at least 224 of 256 channels while under the rotated basis only 3 do; at 250M the coordinate basis needs the entire 512-wide latent for 77 of 768 probes and the rotated basis for none. In these models a large part of the tail-capacity requirement in Section 5.1 is therefore a cost of axis alignment rather than a property of the latent code. A fixed change of variables folded into the projection weights at identical cache bytes is a candidate remedy, but we have not trained or evaluated the router or the packed cache in a rotated basis.

The effect's magnitude does not transfer to a production-trained model. On DeepSeek-V2-Lite's 512-wide MLA latent the same four-basis probe lowers the worst-offset requirement by only 11.3 points, from 78.65% to 67.34% (Section 5.1). An earlier control on a different object agrees: per-head truncation of an ordinary KV cache shows a coordinate-basis overstatement of 1.65-3.22x on our three models but only 1.06-1.30x on GPT-2 and GPT-2-medium over the same 0.05-0.5-nat budgets (`notes/kv_cache_basis_generalization.md`, `notes/low_rank_does_not_mean_compressible.md`). The most economical explanation is that 3,000-8,000-step TinyStories models are unrepresentatively compressible in a rotated basis. That is a caution about this ablation and about what our models can say on their own; it is not a caution about the risk-capacity spectrum, which the production model reproduces.

Which rotation matters much less than rotating at all. Ordering directions by the first-order loss effect is consistently 10-12 points worse than ordering them by plain reconstruction of the latent, at every scale — the worst-offset criterion binds at roughly a quarter of full width, far outside the regime where a linearisation taken at full rank applies. We also tested the frame that provably minimises reconstruction error in the metric the model actually applies to the latent ($A = W_{UK}^\top W_{UK} + W_{UV}^\top W_{UV}$, principal directions of $A^{1/2}\mathbb{E}[cc^\top]A^{1/2}$); it wins by under a point at 30M and 250M and is indistinguishable from plain reconstruction at 122M, with roughly half of all probes tied at every scale. Optimality for squared reconstruction error does not transfer into a reliable advantage under a loss-threshold criterion.

Two qualifications: under rotated bases more probes have a rank curve with any rise (59% to 82% at 30M, 78% to 98% at 250M; by mean violation magnitude the rotated bases are lower at 30M and 122M), and the mean-criterion values for the rotated bases sit on the rank grid's smallest entry and are upper bounds rather than estimates. The ablation's coordinate arm reproduces the Section 5.1 mean and max endpoints and their histograms exactly at every scale (the bootstrap intervals differ slightly because the seeds do). See `notes/basis_dependence_prediction7.md` and `experiments/analyze_basis_dependence.py`.

Reproducibility. Every file path cited in this paper (`experiments/...`, `notes/...`) refers to the supplementary archive, which holds the code, unit tests, result artifacts and dated lab notes. Result artifacts contain checkpoint/data/policy/source hashes and exact starts. A post-hoc audit reconstructs the pre-specified sampler, verifies nonoverlap, derives cache bytes from authenticated checkpoint dimensions, and recomputes all summary statistics. Deterministic training replays reproduce the frozen router tensors, histories, splits, and hyperparameters bit-for-bit. The original cloud commands preceded the later manifest-enforcement arguments; this is disclosed, and the frozen protocol commit plus complete logs and audits bind the original results. Future runs fail unless their manifest inputs match.

7 Conclusion

We formalize token-wise MLA latent width as a basis-dependent operational rate allocation problem: persistent cache bytes are exactly affine in summed token rates (Proposition 1), and upper-tail future-loss risk induces a provably monotone spectrum of safe rates between average and worst-case reuse (Proposition 2). Measured at 30M, 122M, and 250M (an 8x parameter range), this spectrum is nearly scale-invariant in its normalized tail-capacity premium (0.646-0.651 across all three scales) and shows that the mean/tail separation arises from pervasive cancellation across the reuse horizon rather than rare catastrophic tokens — a mechanistic finding that revises the intuitive "rare spike" story and is the paper's most robust contribu-

tion, independent of the router. Both the spectrum and the cancellation signature replicate on the public, production-trained DeepSeek-V2-Lite (premium 0.704; 96.6% of mean-safe probes with an offset above budget), where, unlike in our small models, rotating the latent basis lowers the worst-offset requirement by only 11 points.

The routing contribution requires two axes to describe honestly, both established through pre-registered, audited protocols rather than post-hoc adjustment. First, whether the router beats *random* matched-budget allocation depends on tier granularity as well as scale: it succeeds with a coarse 4-tier grid at 30M and a fine 14-tier grid at 122M, fails with a coarse grid at 250M, and succeeds again at 250M once tier resolution is increased to match 122M's scheme — so the initial appearance of a pure scale effect was substantially a tier-resolution confound in our own design, which we diagnosed and corrected rather than let stand. Second, held to exactly the router's bytes, simple **causal heuristics** (position, token identity, rarity, type) lose to the router 6 times, tie 8 times and win twice across four tested scale/tier-grid configurations (16 comparisons): position and type lose everywhere except 250M-coarse, where both router losses also fall, while a frequency-smoothed per-token identity table ties the router at every other configuration. We report both axes rather than overclaim: the present evidence establishes a formal allocation framework, a scale-consistent risk-capacity spectrum, and a routing method with a real but tier-grid-dependent advantage over random placement and over position/type placement, but no measured advantage over a per-token lookup table — not peak-memory or throughput superiority (a direct T4 benchmark initially showed the packed path 168-201x slower per decode step than full MLA; vectorizing three Python cache-construction loops we identified cut this to roughly 3x at both scales, and making the cache append in place rather than repack its whole history each step cut it further to 2.4x, validated against random-trial correctness cross-checks and bit-identical end-to-end decode output, but a real 2.4x cost remains from the packed path's dense-reconstruction gather), and not a reliably calibrated quality constraint at 122M or 250M-fine. Priority future work is (1) showing whether context beyond the current token can improve placement at all — via joint-rollout-consistent oracle labels, tier-resolution-aware training, or a router given the per-token table as a baseline to improve on — before further scale-up is attempted, (2) computing the per-head key/value projections directly against the ragged packed buffer, to remove the dense-reconstruction gather that now dominates the residual latency cost, and a grouped-tier or fused packed attention kernel to convert the established persistent-byte savings into measured peak-memory and throughput gains against optimized MHA/GQA/MLA/FlashMLA baselines, and (3) replication across more seeds, domains, and larger, more realistically trained checkpoints once (1) is resolved.

References

[1] A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017.

[2] DeepSeek-AI, "DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model," arXiv:2405.04434, 2024.

[3] DeepSeek-AI, "DeepSeek-V3 Technical Report," arXiv:2412.19437, 2024.

[4] T. Ji et al., "Towards Economical Inference: Enabling DeepSeek's Multi-Head Latent Attention in Any Transformer-based LLMs," arXiv:2502.14837, 2025.

[5] "Lossless KV Cache Compression to 2%" (Cross-Layer Latent Attention), arXiv:2410.15252, 2024.

[6] "TransMLA: Multi-Head Latent Attention Is All You Need," arXiv:2502.07864, 2025.

[7] "EG-MLA: Embedding-Gated Multi-head Latent Attention for Scalable and Efficient LLMs," arXiv:2509.16686, 2025.

[8] "CARE: Covariance-Aware and Rank-Enhanced Decomposition for Enabling Multi-Head Latent Attention," arXiv:2603.17946.

[9] "Through the Bottleneck: How Multi-head Latent Attention Separates Content from Position in Language Models," arXiv:2607.23054.